Cités des Métiers et des Compétences
La formation nouvelle génération

OFPPT
La voie de l'avenir

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle
et de la Promotion du Travail

Compte-rendu

Devoir N °2

Prénom : EL HAIRCHE
Nom : MOHAMED
Groupe : DEV106

Année scolaire : 2025/2026

Introduction :

Ce devoir s'inscrit dans le cadre du module de programmation orientée objet en Python. Il a pour but de mettre en pratique les notions fondamentales telles que la création de classes, l'encapsulation, l'héritage et la redéfinition des méthodes.

À travers ce travail, nous avons conçu une application permettant de gérer les personnes au sein d'une entreprise. Cette gestion inclut différents types de personnes, notamment les directeurs et les employés, chacun ayant des caractéristiques spécifiques. Le programme permet d'ajouter, rechercher, supprimer et afficher les informations des personnes.

Question 1:

```python
import re


class Personne:  1 usage
    compteur = 0

    def __init__(self, reference, nom, prenom, age):
        if not self.valider_reference(reference):
            raise ValueError("Référence invalide ! (Format: AA12345)")

        self.__reference = reference
        self.__nom = nom
        self.__prenom = prenom
        self.__age = age

        Personne.compteur += 1
```

Figure1: la création de premier class

Nous avons créé une classe Personne avec des attributs privés : référence, nom, prénom et âge .Un compteur statique a été ajouté pour compter le nombre d'objets créés.

```python
    def get_reference(self):  1 usage (1 dynamic)
        return self.__reference

    def get_nom(self):
        return self.__nom

    def get_prenom(self):
        return self.__prenom

    def get_age(self):
        return self.__age

    # Setters
    def set_nom(self, nom):
        self.__nom = nom

    def set_prenom(self, prenom):
        self.__prenom = prenom

    def set_age(self, age):
        self.__age = age
```

Figure2: La création des définitions (getters/setters : un nom, un prénom, et un âge.)

Les getters et setters sont des méthodes utilisées pour accéder et modifier les attributs privés d'une classe. Le getter permet de lire la valeur d'un attribut, tandis que le setter

permet de la modifier tout en contrôlant les données. Cela garantit la sécurité et l'encapsulation des informations dans le programme.

```
def valider_reference(self, ref):  1 usage
    return re.match( pattern: r'^[A-Z]{2}\d{5}$', ref) is not None


def __str__(self):
    return f"{self.__reference}, {self.__nom}, {self.__prenom}, {self.__age}"


def __eq__(self, other):
    return self.__reference == other.get_reference()
```

Figure3:. Redéfinir la méthode ToString et Equals et ajouter la méthode ValiderReference

Question 2:

```
class Directeur(Personne):
        def __init__(self, reference, nom, prenom, age, direction, fonction):
            super().__init__(reference, nom, prenom, age)
            self.direction = direction
            self.fonction = fonction

        def __str__(self):
            return super().__str__() + f", Direction: {self.direction}, Fonction: {self.fonction}"
```

Figure4: la création de class Directeur

La création de la classe Directeur hérite de la classe Personne et ajoute deux attributs direction et fonction et aussi le constructeur utilise super() pour appeler celui de la classe mère et ona ajouter la méthode **str** est redéfinie pour inclure les informations supplémentaires.

Question 3:

```
class Employe(Personne):
        def __init__(self, reference, nom, prenom, age, tache, service):
                super().__init__(reference, nom, prenom, age)
                self.tache = tache
                self.service = service
```

Figure5: la création de class Employe

La création de la classe Employe hérite également de Personne et elle contient les attributs tâche et service.

```
    def valider_jour_mois_annee(self, date):
        return re.match( pattern: r'^\d{2}/\d{2}/\d{4}$', date) is not None

    def __str__(self):
        return super().__str__() + f", Tâche: {self.tache}, Service: {self.service}"
```

Figure6:. Redéfinir la méthode ToString et ajouter la méthode valider_jour_mois_annee

Ajouter une méthode valider_jour_mois_annee() est définie pour valider le format d'une date (jj/mm/aaaa) à l'aide d'une expression régulière, et la redéfinir de la méthode **str** affiche les informations complètes d'un employé

Question 3:

```
class Entreprise:
    def __init__(self, nom, adresse):
        self.nom = nom
        self.adresse = adresse
        self.personnes = []

    def __str__(self):
        resultat = f"Entreprise: {self.nom}, Adresse: {self.adresse}\n"
        for p in self.personnes:
            resultat += str(p) + "\n"
        return resultat
```

Figure7: la création de class Entreprise

Création de la classe Entreprise permet de gérer une liste de personnes qui contient nom, adresse et une liste de personnes ainsi que la méthode ToString() qui renvoie la description d'une entreprise.

```
    def ajouter_personne(self, personne):
        self.personnes.append(personne)


    def chercher_personne(self, reference):
        for p in self.personnes:
            if p.get_reference() == reference:
                return p
        return None
```

Figure8: ona ajouter deux méthodes

Ona ajputer deux méthodes s'appelle ajouter_personne() qui permet de ajoute une personne à la liste, et la deuxième s'appelle chercher_personne() qui permet recherche une personne par référence.

```
def liste_directeurs(self):
    for p in self.personnes:
        if isinstance(p, Directeur):
            print(p)


def supprimer_personne(self, reference):
    self.personnes = [p for p in self.personnes if p.get_reference() != reference]


def enregistrer_personne(self, fichier):
    with open(fichier, "w") as f:
        for p in self.personnes:
            f.write(str(p) + "\n")
```

Figure9: ajoutation de trois méthodes

One ajouter trois méthodes s'appelle liste_directeurs() qui permet de retourne uniquement les directeurs et la deuxième s'appelle supprimer_personne() qui permet de supprime une personne selon sa référence et troisième et le dernier s'appelle enregistrer_personnes() qui permet de sauvegarde les données dans un fichier texte.

```
Personne
 ├── Directeur
 └── Employe

Entreprise
 └── contient → liste de Personne
```

Figure10: un dessin de schéma representant les classes

Le schéma montre que la classe Personne est la classe principale. Les classes Directeur et Employe héritent de Personne. La classe Entreprise contient une liste de personnes pour les gérer (ajout, recherche, suppression).

Conclusion :

Ce devoir nous a permis de mieux comprendre les concepts fondamentaux de la programmation orientée objet en Python, tels que l'encapsulation, l'héritage et le polymorphisme. À travers la création des classes Personne, Directeur, Employe et Entreprise, nous avons appris à structurer un programme en organisant les données et les comportements de manière claire et logique.

Ce travail nous a également aidés à manipuler les listes d'objets, à implémenter des méthodes de gestion comme l'ajout, la recherche et la suppression, ainsi qu'à valider les données pour garantir leur conformité. De plus, l'utilisation de fichiers pour enregistrer les informations montre une application concrète de ces concepts dans un contexte réel.

Ainsi, ce devoir constitue une base importante pour développer des applications plus complexes et mieux comprendre l'organisation d'un programme orienté objet.